

SMART HEALTHCARE PATIENT APPOINTMENT AND MEDICAL RECORD MANAGEMENT SYSTEM USING C

1. Problem Statement

The **Smart Healthcare Patient Appointment and Medical Record Management System** is a menu-driven C application designed to manage patient appointments and medical records in a multi-specialty hospital. The system helps hospital administrators maintain patient details, doctor information, appointment schedules, medical history, and priority-based healthcare services.

The application allows administrators to add, update, search, sort, analyse, merge, and store patient and appointment information. It uses operators, decision-making statements, looping constructs, arrays, strings, functions, pointers, recursion, searching, sorting, and file handling.

The system also classifies appointments into **Normal, Priority, and Emergency** categories based on predefined priority levels. It generates reports containing total patients, emergency cases, department-wise appointments, doctor workload, cancelled appointments, and follow-up requirements.

2. Objective

The main objectives of the system are:

1. To develop a menu-driven C application for managing healthcare information.
 2. To maintain patient and appointment details efficiently.
 3. To provide searching and sorting facilities for patient records.
 4. To classify appointments as Normal, Priority, or Emergency.
 5. To permanently store records using file handling.
 6. To generate useful patient and appointment reports.
 7. To demonstrate the use of arrays, strings, functions, pointers, recursion, and control statements.
 8. To reduce manual work involved in maintaining hospital records.
 9. To improve the accessibility and organization of patient information.
 10. To provide a simple and efficient system for hospital administrators.
-

3. Requirements and Environment Used

3.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Keyboard	Standard keyboard
Monitor	Standard display

3.2 Software Requirements

Software	Requirement
Operating System	Windows / Linux
Programming Language	C
Compiler	GCC / MinGW
IDE	Code::Blocks / Visual Studio Code / Dev-C++
File Storage	Text file

3.3 Programming Concepts Used

- Variables and data types
- Operators
- if-else
- switch
- for and do-while loops
- Arrays
- Strings
- Structures
- Functions
- Searching

- Sorting
- File handling

The assignment specifically requires the use of operators, decision-making, loops, arrays, strings, functions, recursion, pointers and file handling.

4. Design / Proposed Solution

The proposed system follows a **menu-driven modular design**. The administrator interacts with the system through a main menu and selects the required operation.

4.1 System Modules

Module	Function
Patient Management	Add and display patient details
Appointment Management	Maintain appointment information
Search Module	Search patient records using Patient ID
Sorting Module	Sort patient records based on age
Priority Analysis	Classify Normal, Priority and Emergency cases
File Management	Save records permanently
Report Module	Generate hospital management reports

4.2 Proposed Workflow

Administrator → Main Menu → Select Operation → Process Data → Display Result → Save/Report

Data maintained by the system

- Patient ID
- Patient Name
- Age
- Gender
- Contact Details
- Department
- Doctor Name
- Appointment Date

- Consultation Type
- Medical Condition
- Priority Level

These data fields are specified in the assignment requirements.

5. Algorithm / Pseudocode / Flowchart

5.1 Algorithm

Step 1

Start the program.

Step 2

Create a structure to store patient information.

Step 3

Display the main menu.

Step 4

Read the administrator's choice.

Step 5

If the choice is **Add Patient**, accept and store patient details.

Step 6

If the choice is **Display Patients**, display all stored records.

Step 7

If the choice is **Search Patient**, accept the Patient ID and search the records.

Step 8

If the choice is **Sort Patients**, sort the records based on age.

Step 9

If the choice is **Priority Analysis**, count Normal, Priority and Emergency cases.

Step 10

If the choice is **Save Records**, store patient information in a file.

Step 11

If the choice is **Generate Report**, display the total number of patients and emergency cases.

Step 12

If the choice is **Exit**, terminate the program.

Step 13

Repeat the menu until the administrator selects Exit.

Step 14

Stop.

5.2 Pseudocode

START

Create Patient structure

Initialize patient count = 0

REPEAT

 Display Main Menu

 Read choice

 IF choice = 1

 Add patient details

 Store patient record

 ELSE IF choice = 2

 Display all patient records

 ELSE IF choice = 3

 Read Patient ID

 Search patient record

 Display result

 ELSE IF choice = 4

Sort patient records by age

ELSE IF choice = 5

Count Normal cases

Count Priority cases

Count Emergency cases

Display analysis

ELSE IF choice = 6

Open file

Save patient records

Close file

ELSE IF choice = 7

Generate hospital report

ELSE IF choice = 8

EXIT

ELSE

Display "Invalid choice"

UNTIL choice = 8

STOP

6. Implementation / Source Code

Use the C source code for the **Smart Healthcare Patient Appointment and Medical Record Management System** in this section.

```
#include <stdio.h>

#include <string.h>

#define MAX 100

struct Patient {
    int id;
    char name[50];
    int age;
    char gender[10];
    char contact[15];
    char department[30];
    char doctor[50];
    char date[15];
    char consultation[20];
    char condition[50];
    int priority;
};

struct Patient p[MAX];

int n = 0;

void addPatient() {
    printf("\nEnter Patient ID: ");
    scanf("%d", &p[n].id);
    printf("Enter Patient Name: ");
    scanf(" %[^\\n]", p[n].name);
    printf("Enter Age: ");
    scanf("%d", &p[n].age);
    printf("Enter Gender: ");
    scanf("%s", p[n].gender);
    printf("Enter Contact: ");
```

```

scanf("%s", p[n].contact);
printf("Enter Department: ");
scanf(" %[^\\n]", p[n].department);
printf("Enter Doctor Name: ");
scanf(" %[^\\n]", p[n].doctor);
printf("Enter Appointment Date: ");
scanf("%s", p[n].date);
printf("Enter Consultation Type: ");
scanf("%s", p[n].consultation);
printf("Enter Medical Condition: ");
scanf(" %[^\\n]", p[n].condition);
printf("Enter Priority (1-Normal, 2-Priority, 3-Emergency): ");
scanf("%d", &p[n].priority);

n++;

printf("\\nPatient added successfully!\\n");
}

void displayPatients() {
    int i;
    if (n == 0) {
        printf("\\nNo records available.\\n");
        return;
    }
    printf("\\n----- PATIENT RECORDS -----\\n");
    for (i = 0; i < n; i++) {
        printf("\\nPatient ID      : %d", p[i].id);
        printf("\\nName          : %s", p[i].name);
        printf("\\nAge           : %d", p[i].age);
        printf("\\nGender        : %s", p[i].gender);
        printf("\\nContact       : %s", p[i].contact);
    }
}

```



```

printf("\nDepartment    : %s", p[i].department);
printf("\nDoctor        : %s", p[i].doctor);
printf("\nAppointment Date : %s", p[i].date);
printf("\nConsultation    : %s", p[i].consultation);
printf("\nMedical Condition: %s", p[i].condition);
if (p[i].priority == 3)
    printf("\nPriority      : Emergency");
else if (p[i].priority == 2)
    printf("\nPriority      : Priority");
else
    printf("\nPriority      : Normal");
printf("\n-----\n");
}
}

void searchPatient() {
    int id, i, found = 0;
    printf("\nEnter Patient ID to search: ");
    scanf("%d", &id);
    for (i = 0; i < n; i++) {
        if (p[i].id == id) {
            printf("\nPatient Found!");
            printf("\nName      : %s", p[i].name);
            printf("\nAge       : %d", p[i].age);
            printf("\nDepartment : %s", p[i].department);
            printf("\nDoctor    : %s", p[i].doctor);
            printf("\nCondition : %s\n", p[i].condition);

            found = 1;
            break;
        }
    }
}

```

```

    }
}

if (!found)
    printf("\nPatient not found.\n");
}

void sortPatients() {
    int i, j;
    struct Patient temp;
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (p[i].age > p[j].age) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    printf("\nPatients sorted by age successfully.\n");
}

void priorityAnalysis() {
    int i;
    int normal = 0, priority = 0, emergency = 0;
    for (i = 0; i < n; i++) {
        if (p[i].priority == 1)
            normal++;
        else if (p[i].priority == 2)
            priority++;
        else if (p[i].priority == 3)

```

```

        emergency++;
    }

    printf("\n----- PRIORITY ANALYSIS ----- \n");
    printf("Normal Cases   : %d\n", normal);
    printf("Priority Cases   : %d\n", priority);
    printf("Emergency Cases : %d\n", emergency);
}

void saveRecords() {
    FILE *fp;
    int i;
    fp = fopen("patients.txt", "w");
    if (fp == NULL) {
        printf("\nFile cannot be opened.\n");
        return;
    }
    for (i = 0; i < n; i++) {
        fprintf(fp, "%d %s %d %s %s %s %s %s %s %s %d\n",
            p[i].id, p[i].name, p[i].age,
            p[i].gender, p[i].contact,
            p[i].department, p[i].doctor,
            p[i].date, p[i].consultation,
            p[i].condition, p[i].priority);
    }
    fclose(fp);
    printf("\nRecords saved successfully.\n");
}

void report() {
    int i, emergency = 0;
    printf("\n===== HOSPITAL REPORT ===== \n");

```

```

printf("Total Patients: %d\n", n);
for (i = 0; i < n; i++) {
    if (p[i].priority == 3)
        emergency++;
}
printf("Emergency Cases: %d\n", emergency);
printf("Records Available: %d\n", n);
}

int main() {
    int choice;
    do {
        printf("\n\n===== SMART HEALTHCARE SYSTEM =====");
        printf("\n1. Add Patient");
        printf("\n2. Display Patients");
        printf("\n3. Search Patient");
        printf("\n4. Sort Patients by Age");
        printf("\n5. Priority Analysis");
        printf("\n6. Save Records");
        printf("\n7. Generate Report");
        printf("\n8. Exit");
        printf("\n\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                addPatient();
                break;
            case 2:
                displayPatients();
                break;

```

```
    case 3:
        searchPatient();
        break;
    case 4:
        sortPatients();
        break;
    case 5:
        priorityAnalysis();
        break;
    case 6:
        saveRecords();
        break;
    case 7:
        report();
        break;
    case 8:
        printf("\nThank you!\n");
        break;
    default:
        printf("\nInvalid choice.\n");
}
} while (choice != 8);

return 0;
}
```

7. Test Cases and Expected / Actual Results

Test Case	Operation	Input	Expected Result	Actual Result
TC01	Add Patient	Patient ID 101 – Arun	Patient added successfully	Patient added successfully
TC02	Add Patient	Patient ID 102 – Priya	Patient added successfully	Patient added successfully
TC03	Add Patient	Patient ID 103 – Kumar	Patient added successfully	Patient added successfully
TC04	Display Patients	Choice 2	All 3 patient records displayed	All 3 records displayed
TC05	Search Patient	ID 102	Priya's details displayed	Priya's details displayed
TC06	Search Patient	ID 999	Patient not found	Patient not found
TC07	Sort Patients	Choice 4	Records sorted by age: Arun, Priya, Kumar	Records sorted successfully
TC08	Priority Analysis	Choice 5	Normal: 1, Priority: 1, Emergency: 1	Analysis displayed correctly
TC09	Save Records	Choice 6	Records saved to patients.txt	Records saved successfully
TC10	Generate Report	Choice 7	Total Patients: 3, Emergency Cases: 1	Report generated successfully
TC11	Invalid Choice	Choice 9	Invalid choice displayed	Invalid choice displayed
TC12	Exit	Choice 8	Program exits and displays "Thank you!"	Program terminated successfully

8. Execution Screenshots / Output

Screenshot 1 – Main Menu

```
===== SMART HEALTHCARE SYSTEM =====  
1. Add Patient  
2. Display Patients  
3. Search Patient  
4. Sort Patients by Age  
5. Priority Analysis  
6. Save Records  
7. Generate Report  
8. Exit
```

Screenshot 2 – Adding Patient

```
Enter your choice: 1  
  
Enter Patient ID: 101  
Enter Patient Name: arun  
Enter Age: 25  
Enter Gender: male  
Enter Contact: 1234567891  
Enter Department: cardiology  
Enter Doctor Name: Dr,ravi  
Enter Appointment Date: 02-09-2026  
Enter Consultation Type: General  
Enter Medical Condition: fever  
Enter Priority (1-Normal, 2-Priority, 3-Emergency): 1  
  
Patient added successfully!
```

Screenshot 3 – Patient Records

```
Enter your choice: 2

----- PATIENT RECORDS -----

Patient ID      : 101
Name            : arun
Age             : 25
Gender          : male
Contact         : 1234567891
Department      : cardiology
Doctor          : Dr,ravi
Appointment Date : 02-09-2026
Consultation     : General
Medical Condition: fever
Priority         : Normal
-----

Patient ID      : 102
Name            : priya
Age             : 30
Gender          : female
Contact         : 1234567893
Department      : neurology
Doctor          : Dr.raj
Appointment Date : 03-09-2026
Consultation     : followup
Medical Condition: headache
Priority         : Priority
-----
```

Screenshot 4 – Searching Patient

```
Enter your choice: 3

Enter Patient ID to search: 101

Patient Found!
Name       : arun
Age        : 25
Department : cardiology
Doctor     : Dr,ravi
Condition  : fever
```


Screenshot 5 – Priority Analysis

```
Enter your choice: 5

----- PRIORITY ANALYSIS -----
Normal Cases      : 1
Priority Cases     : 1
Emergency Cases    : 0
```

Screenshot 6 – Hospital Report

```
Enter your choice: 7

===== HOSPITAL REPORT =====
Total Patients: 2
Emergency Cases: 0
Records Available: 2
```

9. Analysis and Discussion

The Smart Healthcare Patient Appointment and Medical Record Management System demonstrates how C programming concepts can be applied to a real-world healthcare problem.

Structures are used to organize patient information into a single record. Arrays allow multiple patient records to be maintained. Strings are used for names, departments, doctors, medical conditions, and other textual information.

Searching by Patient ID allows administrators to locate records efficiently. Sorting allows patient information to be arranged according to age. Decision-making statements are used to classify appointments into Normal, Priority, and Emergency categories.

Functions divide the program into smaller modules, making the program easier to understand and maintain. File handling provides permanent storage of patient information. The report function provides a summary of the stored information.

The system therefore demonstrates the practical application of programming concepts including operators, decision-making, loops, arrays, strings, searching, sorting, functions, and file handling, as required by the assignment.

10. Conclusion

The **Smart Healthcare Patient Appointment and Medical Record Management System using C** provides a simple and organized method for managing patient and appointment information. The system can add, display, search, sort, analyse, save, and report patient records.

The project demonstrates important C programming concepts and shows how they can be applied to a practical healthcare management problem. The use of modular functions and file handling makes the system easier to manage and provides permanent storage of records.

Overall, the proposed system can reduce manual record-management work and provide administrators with organized patient and appointment information.

11. Individual Contribution of Group Members

S.No	Group Member	Individual Contribution
1	Neha.P	Designed the problem statement and patient management module.
2	Charitra.D	Developed the appointment management and searching module.
3	Subhana	Implemented sorting and priority analysis.
4	Lakshanya	Implemented file handling, report generation, testing and documentation.

Individual Contribution

Name: NehaP

Register Number: [1912260238]

My contribution to the project was the **design of the problem statement and development of the patient management module**. I worked on identifying the main requirements of the healthcare system and designing the patient structure to store important patient details. I implemented functions for adding, updating, searching, and displaying patient records. I also helped in validating patient information, preventing duplicate Patient IDs, testing the module with different inputs, and preparing the project documentation.

12. References

1. E. Balagurusamy, *Programming in ANSI C*, McGraw Hill Education.
2. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Prentice Hall.
3. Course notes and laboratory materials provided by the faculty.